

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

# A Privacy-Preserving Password Strength and Breach Detection Tool

Combining Entropy, Pattern, Dictionary and k-Anonymity Breach Checks in One Open-Source System


**By**

**Faizan Sawal**

Student ID: G00485811

Supervisor: **Dr. Owen Foley**


October 26, 2026

Source code: `https://github.com/FaizanS27/pwbreach-tool`

M.Sc. in Computing

## Masters Thesis

Submitted in fulfilment of the requirements for the award of **Master of Science in Computing** to the Department of Computer Science & Applied Physics, Atlantic Technological University (ATU), Galway.

# Contents


**1 Introduction** **1**
- 1.1 Context . . . 1
- 1.2 Objectives . . . 1
- 1.3 Research Questions . . . 1
- 1.4 Scope . . . 2
- 1.5 Thesis Structure . . . 2

**2 Research Methodology** **3**
- 2.1 Software Development Methodology . . . 3
- 2.2 Research Methodology . . . 3
  - 2.2.1 Measurement-driven problem solving . . . 3
  - 2.2.2 Comparative evaluation . . . 3
  - 2.2.3 User study (RQ3) . . . 3
  - 2.2.4 Threats to validity . . . 4

**3 Literature Review** **5**
- 3.1 Password Strength Estimation . . . 5
- 3.2 Compromised Credential Checking . . . 5
- 3.3 Dictionaries, Leaks and Probabilistic Membership . . . 5
- 3.4 Gap and Positioning . . . 6

**4 System Design** **7**
- 4.1 Architecture Overview . . . 7
- 4.2 Verdict Priority . . . 7
- 4.3 Design Decisions . . . 7
  - 4.3.1 Check ordering and early exit . . . 7
  - 4.3.2 Bloom filter versus exact structures . . . 9
- 4.4 Privacy Design . . . 9
- 4.5 Scaling to BreachCompilation . . . 9
  - 4.5.1 Design decision: streaming read . . . 9
  - 4.5.2 Design decision: Bloom filter . . . 9
  - 4.5.3 Encoding . . . 9
- 4.6 Web Application . . . 9
- 4.7 Browser Extension . . . 10
- 4.8 Privacy, Security and Ethical Considerations . . . 10
- 4.9 Implementation and Availability . . . 11

**5 System Evaluation** **12**
- 5.1 Performance (RQ2, Objective O2) . . . 12
- 5.2 Defects Found Through Testing . . . 12
- 5.3 Demonstrating the Value of BreachCompilation . . . 13

5.4 Real-World Validation of the Extension . . . . . 13
5.5 Accuracy against zxcvbn (RQ1) . . . . . 13
5.6 Usability (RQ3) . . . . . 14
5.7 Limitations and Threats to Validity . . . . . 15

**6 Conclusion** **16**
6.1 Future Work . . . . . 16

**A Source Code Repository** **17**

**Bibliography** **18**

# List of Figures

4.1 System architecture and check execution order. . . . . 8
4.2 The web application report card for a compromised password. . . . . 10
4.3 Extension showing a live “Strong” badge on a test sign-up page. . . . . 10

5.1 Extension marking a weak password as Compromised on Amazon’s sign-up page. 13

# List of Tables

2.1 Work-package structure of the project. . . . . 3

3.1 Comparison with recent research. . . . . 6

4.1 The five checks. . . . . 7

5.1 Speed before and after the Bloom filter. . . . . 12
5.2 Engineering problems and their resolutions. . . . . 12
5.3 Results on the 25-password test set . . . . . 14
5.4 Limitations and threats to validity. . . . . 15

# Abstract

Most password-strength meters judge how a password *looks*. Attackers, however, succeed with passwords that have already *leaked*. The open tools I found cover only one of these two views, and I could not find a freely available tool that combines entropy analysis, pattern matching, a common-password dictionary and privacy-preserving breach checking in one result. This thesis describes how I designed, built and tested such a tool.

The tool runs five checks in a fixed order: a Have I Been Pwned (HIBP) k-anonymity query, the RockYou dictionary, a Bloom filter built from the BreachCompilation dataset, Shannon entropy, and zxcvbn. It returns one verdict. A confirmed breach always outranks a common password, which outranks a guessable pattern, which outranks a "Strong" rating. BreachCompilation is about 10 GB and 1 billion lines long, and it caused three problems: the file was too large for memory, a worst-case lookup took 171.10 s, and a UTF-8 bug corrupted non-English passwords. Reading the file in small pieces fixed the memory problem. A Bloom filter cut the lookup time to under 0.001 s, more than 170,000 times faster, at a false-positive rate of about 1 in 1000 and with no false negatives. Setting the correct encoding fixed the bug. The checks are available through a Flask web application, which also shows an estimated crack time and suggests a stronger password, and through a Chrome extension that I tried on a live Amazon sign-up page. The code, a small demo dataset and unit tests are public, so the tool can be run without the large files.

I tested the tool on 25 passwords. It flagged all 18 that were known to be breached or common. zxcvbn on its own rated 4 of those 18 as acceptable, including one that only the BreachCompilation check found. The test set is small and I chose it by hand, so this shows that structure-only scoring can miss leaked passwords, but it does not show how often. The test also revealed a weakness: four predictable passwords made of a name or place plus a year were rated Strong, because zxcvbn scores them highly. The usability study (RQ3) needs ethics approval, so it was outside the scope of this work (Chapter 5). The claim that the integrated tool gives users better guidance is therefore not tested here, and I leave it as future work.

# Acknowledgments

I would like to thank my supervisor, Dr. Owen Foley, for his guidance, feedback and patience throughout this project. I am grateful to the Department of Computer Science and Applied Physics at Atlantic Technological University Galway for the facilities and support that made this research possible. Finally, I would like to thank my family for their support and encouragement throughout this MSc.

# Chapter 1

# Introduction

## 1.1 Context

Stolen or guessed passwords are still one of the most common ways accounts get compromised. Most people are guided by a password-strength meter, which estimates how hard a password would be to guess from its structure. This guidance leaves out two things. First, a password that looks complex but already appears in a public breach is effectively known to attackers, so *whether a password has leaked matters more than how it looks.* Second, checking a password against a breach service must not expose the password itself, so any usable checker has to protect the string being tested.

Existing research treats these two concerns separately. Strength estimation, for example zxcvbn [1], models how guessable a password is. Compromised-credential checking, for example Pwned Passwords [3] and MIGP [7], models whether a password has been exposed. I could not find an open, easy-to-use tool that gives both answers in one verdict, and that gap is the motivation for this thesis.

## 1.2 Objectives

The overall goal is an open-source password checker that combines entropy analysis, pattern matching, dictionary checking and privacy-preserving breach detection into one simple result. I set three specific objectives:

**O1.** Design one assessment engine whose checks are ordered so that fast checks run first and slow checks are skipped once the verdict is certain.

**O2.** Scale the engine to a billion-line breach dataset (BreachCompilation), keeping lookups fast enough for real-time use while protecting the user's privacy.

**O3.** Put the engine where people actually type passwords, through a web application and a browser extension, and evaluate how accurate and useful it is compared with structure-only tools.

## 1.3 Research Questions

One main question and three supporting questions guide the project. **Main question:** Can a single, open-source tool that combines entropy analysis, pattern matching, dictionary checking and privacy-preserving breach detection give users more accurate and more useful guidance than existing structure-only tools?

- **RQ1.** How accurately does the integrated tool identify passwords that are both structurally weak and present in known breach databases, compared with zxcvbn on its own?
- **RQ2.** Does the HIBP k-anonymity prefix query protect the user's privacy without adding enough delay to spoil the real-time experience?
- **RQ3.** Does breach-aware, actionable feedback lead users to choose stronger passwords than a structure-only meter does?

## 1.4 Scope

The tool runs locally. The web application and the server behind the extension both run on my own machine and are not hosted online. The RockYou list I use is a sample of 12,669 passwords, not the full list of about 14 million. The breach check finds exact matches only. RQ3 needs a user study that requires ethics approval, and I have not yet run it. I discuss these limits in Section 5.7. The source code and a small demonstration dataset are available at `https://github.com/FaizanS27/pwbreach-tool`.

## 1.5 Thesis Structure

Chapter 2 describes the development and research methods. Chapter 3 reviews the literature and places the project in it. Chapter 4 presents the system design. Chapter 5 evaluates the system against the objectives and research questions. Chapter 6 concludes and lists future work.

# Chapter 2

# Research Methodology

## 2.1 Software Development Methodology

I developed the tool in work packages, shown in Table 2.1. Each package produced something I could demonstrate, and I fixed problems found at one stage (memory, speed, encoding) before the next stage depended on them.

| WP | Name | Purpose |
|---|---|---|
| WP1 | Foundations | Environment and tooling set up. |
| WP2 | Core library | Five checks built, tested and sped up. |
| WP3 | Web app | Flask report-card page and Chrome extension. |
| WP4 | Evaluation | Comparison testing against zxcvbn and HIBP; growing the test set. |
| WP5 | Write-up | Literature review and dissertation chapters. |

Table 2.1: Work-package structure of the project.

## 2.2 Research Methodology

### 2.2.1 Measurement-driven problem solving

Every engineering claim in this thesis comes from a measurement, not an assumption. I timed lookups with a script and again live in the browser. I found the encoding bug by reading the data back and checking it, instead of assuming it had been saved correctly.

### 2.2.2 Comparative evaluation

For RQ1 I compared the integrated tool with zxcvbn alone and, as a second baseline, with a Have I Been Pwned lookup alone. The test set has 25 passwords. It mixes passwords known to be breached, passwords built to look strong but be predictable, random strong passwords, and passwords taken from the BreachCompilation file. My early test passwords were always caught by HIBP before BreachCompilation could contribute anything. I therefore wrote a script that searches BreachCompilation for passwords HIBP does not know, so that I could show the value of the large dataset instead of guessing at it. The test set is small and I put it together by hand, which limits what it can show (Section 5.7).

### 2.2.3 User study (RQ3)

RQ3 calls for a usability study that compares the passwords people choose under breach-aware feedback and under a structure-only meter. Such a study needs ethics approval, so it was outside

the scope of this work and is left as future work.

### 2.2.4 Threats to validity

The comparison set is small, the RockYou list is only a sample, I tested the extension on only a few pages, and the Bloom filter has a small false-positive rate. I come back to each of these in Section 5.7.

# Chapter 3

# Literature Review

## 3.1 Password Strength Estimation

Early strength meters relied on simple composition rules such as length and character classes, and rule-based meters estimate real guessability poorly [6, 12]. zxcvbn [1] replaced these rules with pattern-based estimation (dictionary words, keyboard walks, dates, repeats) and is now a widely used baseline. Wang et al. [6] evaluated 12 meters on 14 password datasets and found that none of the leading meters is better than the others in every scenario. Pattern-based meters such as zxcvbn did comparatively well in their online-guessing tests. This supports combining several checks instead of trusting one estimator. Other work models how people really choose passwords. Weir et al. [10] used probabilistic context-free grammars to guess passwords, and Melicher et al. [11] showed that neural networks often guess more effectively than grammar-based and Markov models and can be made small enough to run in the browser. Ur et al. [12] tested a data-driven meter and found that its detailed feedback helped people create more secure passwords. This is why RQ3 studies actionable, breach-aware feedback.

## 3.2 Compromised Credential Checking

Pwned Passwords [3] lets a client test a password against a very large breach collection using k-anonymity [4]. Only a short prefix of the password's hash is sent, and the final matching is done locally, so the password never leaves the client. Pal et al. [7] (MIGP) go further and detect *modified* versions of a leaked password, not just exact matches. Thomas et al. [13] describe a deployed breach-alerting system at large scale, and Li et al. [14] analyse the security of protocols for checking compromised credentials, including what prefix-based queries can leak. Kim et al. [8] (PassREfinder) address a different problem: they predict credential-stuffing risk across websites, which is mainly useful to site operators and not to individual users.

## 3.3 Dictionaries, Leaks and Probabilistic Membership

Large password collections, including leaked ones such as RockYou, have long been used to study how people choose passwords [2, 9]. Exact lookup in a billion-line collection costs a lot of memory or time. Bloom filters [5] accept a small, controllable false-positive rate in return for fast membership tests with no false negatives, and they are used in production systems for similar large-set problems.

## 3.4 Gap and Positioning

Table 3.1 summarises the comparison. None of the work I surveyed combines pattern checking, dictionary checking and privacy-preserving breach checking in one open tool that is available at the point where the password is typed. The contribution of this project is the integration, the performance engineering and the delivery. It is not a new cryptographic or estimation technique.

| **Paper** | **What it does** | **Relation to this work** |
|---|---|---|
| Wang et al., 2023 | Compares 12 meters on 14 datasets; none is best in every scenario | Supports combining several checks |
| Pal et al., 2022 (MIGP) | Detects modified versions of leaked passwords | More advanced on this point; a limitation of this work |
| Kim et al., 2024 (PassREfinder) | Predicts website credential-stuffing risk | Different problem, aimed at site operators |

Table 3.1: Comparison with recent research.

# Chapter 4

# System Design

## 4.1 Architecture Overview

People can use the tool in two ways, through a Flask web page or through a Chrome browser extension. Both send the password to the same checking engine (Figure 4.1). The engine runs five checks and returns one verdict. The checks are ordered so that fast ones run first and slower ones are skipped once the answer is certain (Table 4.1).

| **Check** | **Function** | **When it runs** |
|---|---|---|
| HIBP | Checks privately whether the password has leaked before | First (fast, network-based) |
| RockYou | Checks against common passwords (in-memory hash set) | Skipped if HIBP found a breach |
| BreachCompilation | Checks against 1 billion+ leaked passwords (Bloom filter) | Skipped if HIBP or RockYou found a match |
| Shannon entropy | Measures how random the password looks | Always |
| zxcvbn | Checks for common guessable patterns | Always |

Table 4.1: The five checks.

## 4.2 Verdict Priority

The final verdict follows a fixed order of importance: **Compromised** (a real breach) > **Weak (Common)** > **Weak (Predictable)** > **Strong**. A password is marked Strong only if none of the earlier conditions applies. I chose this order because the literature says that exposure matters more than appearance.

## 4.3 Design Decisions

### 4.3.1 Check ordering and early exit

The HIBP check runs first because a confirmed breach is the most serious finding and settles the verdict on its own. The slower BreachCompilation check is reached only if the cheaper checks have not already found a breach. Entropy and zxcvbn always run, because they give the explanation shown to the user when no breach is found.

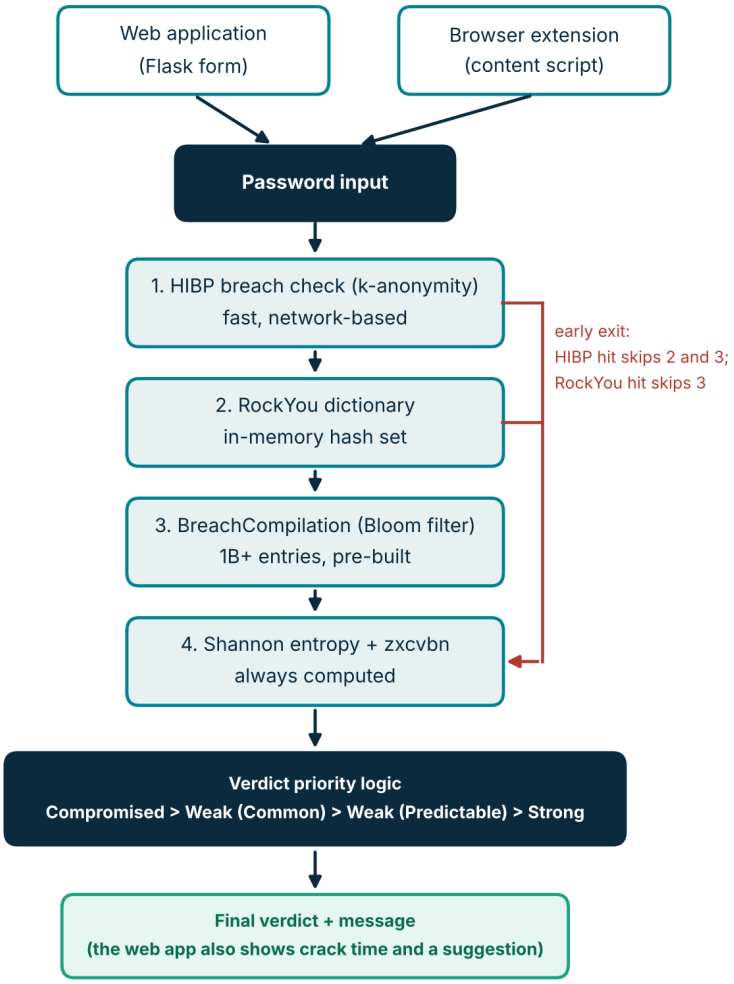



Figure 4.1: System architecture and check execution order.

### 4.3.2 Bloom filter versus exact structures

I considered three ways to test membership in a 10 GB, 1-billion-line list: an in-memory hash set, a streaming scan of the file, and a probabilistic filter. The hash set does not fit in memory. The streaming scan fits but took 171.10 s in the worst case. The Bloom filter gives near-instant lookups from a structure that is loaded once, when the server starts. It costs a one-off build time of about six hours and a false-positive rate of about 0.1%. For a password checker I accept that trade. A rare unnecessary warning does far less harm than a missed breach, and a Bloom filter cannot miss one.

## 4.4 Privacy Design

The HIBP check uses k-anonymity. Only the first five characters of the password's hash are sent, so the real password never leaves the machine. The RockYou comparison uses hashed passwords, not plain text.

## 4.5 Scaling to BreachCompilation

### 4.5.1 Design decision: streaming read

Loading the 10 GB file into memory, as I had done for RockYou, risked using up all the memory. My first design read the file from disk in small pieces. This fixed the memory problem but not the speed.

### 4.5.2 Design decision: Bloom filter

Streaming gave a worst-case lookup of 2 min 51 s, which is far too slow for interactive use. I switched to a Bloom filter. Building it takes one pass over the 1 billion+ lines, about 6 hours, run once in the background. The result is loaded once when the server starts and answers membership queries almost instantly. The trade-off is a false-positive rate of about 1 in 1000. False negatives cannot occur.

### 4.5.3 Encoding

I read and wrote all data as UTF-8, so that non-English passwords such as Cyrillic ones are handled correctly.

## 4.6 Web Application

The Flask application has a page for people and a plain-data endpoint (`/api/check`), so that the extension can use the same engine. The page is a report card (Figure 4.2). It shows the verdict with a colour-coded border (green Strong, orange Weak, red Compromised), an estimated time to crack the password, and one card for each check: entropy, zxcvbn pattern score, HIBP, RockYou and BreachCompilation. A check that was skipped because an earlier check had already settled the verdict appears in grey as "Skipped". For any password that is not Strong, the page offers a randomly generated 16-character replacement, made with Python's cryptographically secure `secrets` module, and a button to copy it. A dark-mode switch is remembered between visits. If the Have I Been Pwned service cannot be reached, the page shows a plain error message and the endpoint returns HTTP 503 instead of failing. The RockYou list and the Bloom filter are loaded once when the server starts, not on every request, which keeps checks fast. The crack-time figure is zxcvbn's estimate for an offline attacker using a slow hash (10,000 guesses per second).

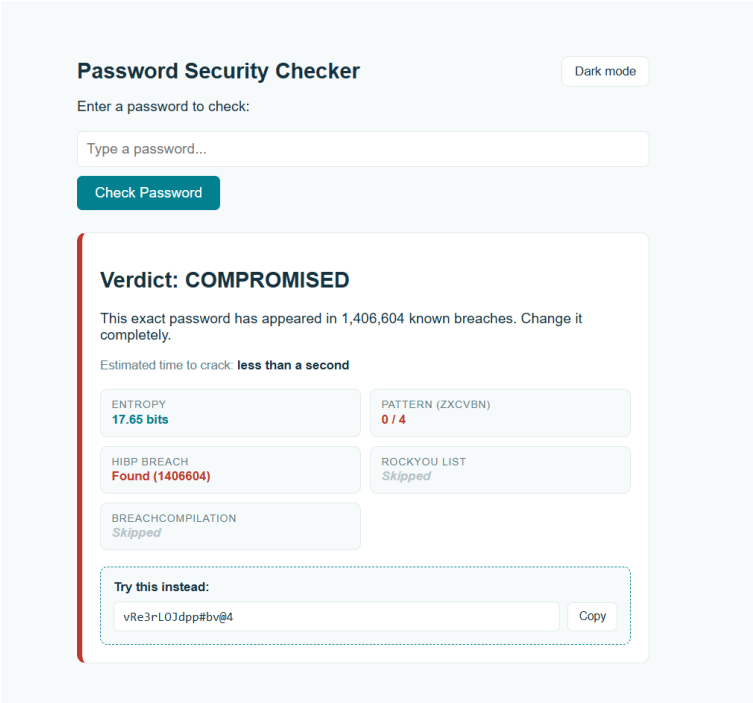



Figure 4.2: The web application report card for a compromised password.

## 4.7 Browser Extension

Few people will visit a separate site to test a password before signing up somewhere else, so I built a Chrome extension that works in place (Figure 4.3). It watches the page for password fields, including ones added after the page loads. It waits 500 ms after typing stops, to avoid sending a request for every keystroke, sends the password to the engine, and shows the result as a small coloured badge next to the field.

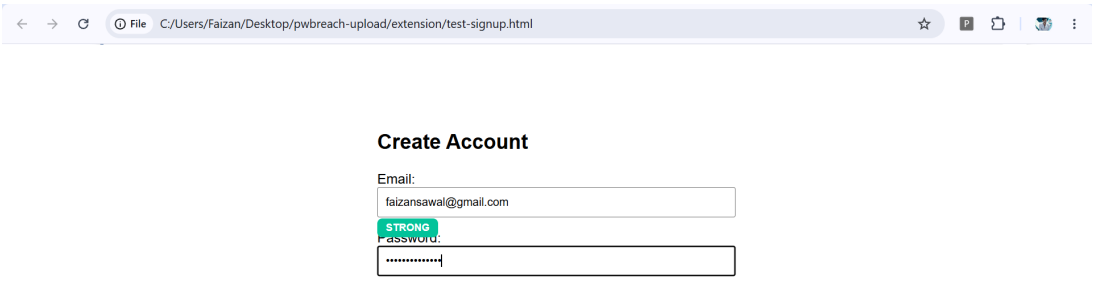



Figure 4.3: Extension showing a live “Strong” badge on a test sign-up page.

## 4.8 Privacy, Security and Ethical Considerations

The tool handles passwords, so privacy is part of the design. The HIBP query sends only a short hash prefix, so the full password is never given to the external service. In this proof of concept the browser extension sends the typed password to the checking engine on my own machine, so the password does not leave the computer. A hosted version would change this. The password would travel to a remote server, which would need encrypted transport (TLS), a policy of not storing or logging submitted passwords, and ideally a k-anonymity step moved into the extension so that the server never sees the password at all. These requirements must be met before any

public deployment.

A user study for RQ3 would ask participants to choose passwords. It needs ethics approval before it starts, participants should be asked to use realistic but non-reused passwords, and no real passwords should be stored.

## 4.9 Implementation and Availability

The system is written in Python (tested with version 3.12) and published at `https://github.com/FaizanS27/pwbreach-tool`. It has four main parts: an engine module that implements the five checks and the verdict priority, a Flask application that serves the web page and the JSON endpoint, a Chrome extension (Manifest V3) that calls the endpoint, and scripts that build the Bloom filter and run the evaluation. Library versions are pinned in a requirements file.

BreachCompilation and RockYou cannot be redistributed, and BreachCompilation is about 10 GB. The repository therefore includes a small synthetic demonstration dataset. The engine switches to it automatically, and prints a "demo mode" warning, whenever the real files are missing. This lets a reader run the web application and the tests on an ordinary laptop in a few minutes. Results that depend on the real datasets, such as the measurements in Section 5.1, can be reproduced only with the full files.

Eight automated unit tests cover the verdict priority, the Bloom filter (no false negatives on the demonstration list) and the JSON endpoint, including how it handles an empty password. The tests need no internet connection, because the Have I Been Pwned call is replaced by a test double. All eight pass.

# Chapter 5

# System Evaluation

## 5.1 Performance (RQ2, Objective O2)

Table 5.1 shows lookup times before and after the Bloom filter, measured with a timing script and by testing live in the browser.

| Check | Before | After |
|---|---|---|
| Worst-case BreachCompilation lookup | 171.10 s | <0.001 s |
| RockYou lookup | Instant already | No change |
| In the actual browser | ∼3 minutes | Instant |

Table 5.1: Speed before and after the Bloom filter.

This is roughly a 170,000-fold speed-up. I also looked up six passwords known to be in BreachCompilation and one made-up password. All six were found and the made-up one was not, and each lookup took 0.000 s at the timer's resolution of one millisecond. RQ2 is therefore only partly answered. The privacy property comes from the design, because only a hash prefix is sent and response padding is switched on. The local checks are shown to be fast. However, the delay of the HIBP network query itself was not measured in this work. It depends on the user's connection and on the service, so I treat it as a limitation and as future work. All timings come from one machine and one network connection, so they show the order of magnitude, not a general figure.

## 5.2 Defects Found Through Testing

Testing found four problems, and I fixed each one with evidence (Table 5.2).

| Problem | Cause | Resolution |
|---|---|---|
| Memory | 10 GB file could not be loaded as RockYou was | Read from disk in small pieces |
| Speed | Disk reads took 171.10 s worst case | Bloom filter, <0.001 s lookups |
| Data corruption | Cyrillic passwords garbled; file read with the wrong text format | Read as UTF-8, matching how it was saved |
| Service outage | If Have I Been Pwned could not be reached, the web page crashed with an error | Added a timeout and status check; the page shows a plain message and the endpoint returns HTTP 503 |

Table 5.2: Engineering problems and their resolutions.

Eight automated unit tests also cover the engine and the endpoint (Section 4.9). They check

each verdict in the priority order, confirm that no password from the demonstration Bloom filter's source list is missed, and confirm that an empty request is rejected. All of them pass.

## 5.3 Demonstrating the Value of BreachCompilation

My early test passwords were caught by HIBP before BreachCompilation could contribute. I wrote a script that searches BreachCompilation for passwords that HIBP does not know. Its first result was a 32-character hexadecimal string. That is a password hash, not something a person typed, so I added a filter on length and character pattern to remove such entries. The script then found a password that is in BreachCompilation but not in HIBP, and I added it to the test set. In the final comparison (Section 5.5), the integrated tool rates it as a common password, while a HIBP lookup alone reports it as clean and zxcvbn gives it a score of 4 out of 4. This is direct evidence that the large dataset can identify a leaked password that the other two checks miss. It is one example found by a targeted search. It shows that the effect exists, not how often it happens. Estimating the frequency would need a random sample of BreachCompilation passwords, which I list as future work.

## 5.4 Real-World Validation of the Extension

I tested the extension on a page I built for the purpose and on Amazon's real account-creation page, where it correctly marked a weak password as Compromised (Figure 5.1). The web page also flagged tested passwords found in breach data as Compromised (Figure 4.2).

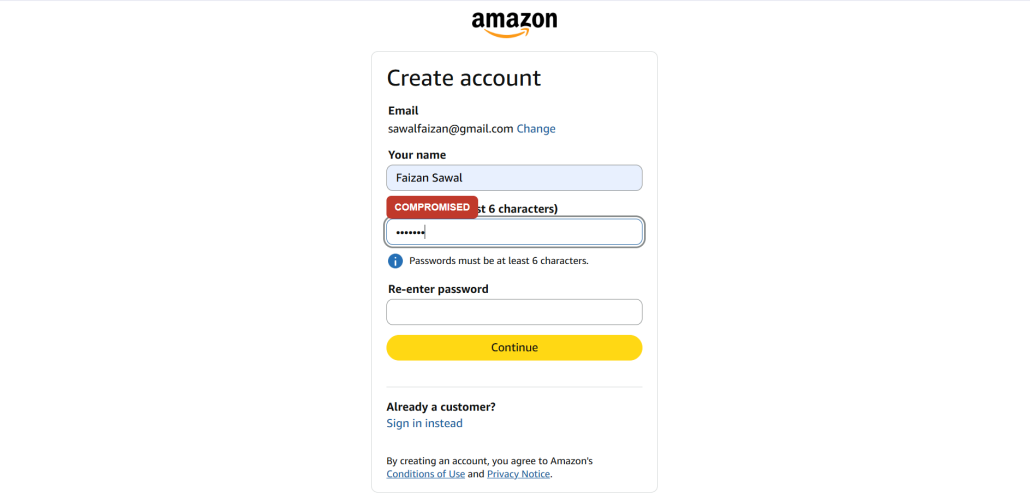



Figure 5.1: Extension marking a weak password as Compromised on Amazon's sign-up page.

## 5.5 Accuracy against zxcvbn (RQ1)

I ran each of the 25 test passwords through zxcvbn alone, through a HIBP lookup alone, and through the integrated engine, using the RockYou sample and the BreachCompilation Bloom filter. I count a password as *breached or common* when the integrated engine returns Compromised or Weak (Common). I treat zxcvbn alone as rating a password acceptable when its score is 3 or 4 (Table 5.3).

Eighteen of the 25 passwords were breached or common: 17 were found by HIBP and one only by BreachCompilation. The integrated tool flagged all 18, as it is designed to. zxcvbn alone rated **4 of the 18 (22%)** as acceptable: `Tr0ub4dor&3` (score 4), `Faizan@12345` (3),

`cfvjqkjdrf105` (4) and the password found only in BreachCompilation (4). Overall, zxcvbn alone rated 15 of the 25 passwords as weak, and the integrated tool flagged 19, which is those 15 plus the 4. One more password, `Zxcvbn1234Qwerty`, was flagged for its pattern and is in no breach list. HIBP alone missed it, but zxcvbn alone caught it. Compared with HIBP alone, the integrated tool added two detections, the pattern-based one and the BreachCompilation-only one. Compared with zxcvbn alone, it added four.

These numbers need careful reading. The set is small and I assembled it by hand, and most of the breached passwords were chosen because they were breached. The result therefore does not estimate how often structure-only meters miss leaked passwords in general. It does show that they can. `Tr0ub4dor&3`, a well-known example of a password that looks strong, scored 4 out of 4 and still appears in breach data.

The test also showed a weakness that the integrated tool inherits. Four passwords that follow a predictable pattern of a name or place plus a year (`Winter2031Ireland`, `Galway2031!`, `MyDogRex2031`, `Atu-Student-2031`) are rated Strong, because zxcvbn scores them 3 or 4 and they are in no breach list. On guessable patterns the integrated tool is only as good as zxcvbn. What it adds is the breach evidence.

| Password | zxcvbn | HIBP | Integrated verdict |
|---|---|---|---|
| `password` | 0 | Flagged | Compromised |
| `123456` | 0 | Flagged | Compromised |
| `qwerty` | 0 | Flagged | Compromised |
| `letmein` | 0 | Flagged | Compromised |
| `iloveyou` | 0 | Flagged | Compromised |
| `Tr0ub4dor&3` | **4** | Flagged | Compromised |
| `correct-horse-battery-staple` | 4 | Clean | Strong |
| `Xk9$mQ2vL7Tz#Random2026` | 4 | Clean | Strong |
| `P@ssw0rd123!` | 2 | Flagged | Compromised |
| `Summer2026!` | 2 | Flagged | Compromised |
| `Faizan20` | 2 | Flagged | Compromised |
| `Winter2031Ireland` | 4 | Clean | Strong |
| `Galway2031!` | 3 | Clean | Strong |
| `MyDogRex2031` | 4 | Clean | Strong |
| `Zxcvbn1234Qwerty` | 2 | Clean | Weak (Predictable) |
| `Atu-Student-2031` | 4 | Clean | Strong |
| `Faizan@12345` | **3** | Flagged | Compromised |
| `Sawal123` | 2 | Flagged | Compromised |
| `teamochapa` | 2 | Flagged | Compromised |
| `cfvjqkjdrf105` | **4** | Flagged | Compromised |
| `brantner1` | 2 | Flagged | Compromised |
| `nshmotherlode` | 2 | Flagged | Compromised |
| `nikoz581` | 2 | Flagged | Compromised |
| `kbvgjgj` | 1 | Flagged | Compromised |
| `890333665189` + 3 Cyrillic letters | **4** | Clean | Weak (Common) |

Table 5.3: Results on the 25-password test set. Bold zxcvbn scores mark the four breached or common passwords that zxcvbn alone rates as acceptable. The last password was found only through BreachCompilation. `teamochapa`, `cfvjqkjdrf105`, `brantner1`, `nshmotherlode`, `nikoz581`, `kbvgjgj` and the last entry are real BreachCompilation entries with no account details attached, used only as test cases.

## 5.6 Usability (RQ3)

*Status: not carried out.* The study needs ethics approval (Section 4.8), so it was left out of this work. This thesis therefore makes no claim about whether breach-aware feedback improves

password choice.

## 5.7 Limitations and Threats to Validity

Table 5.4 lists the main limits of this work.

| Limitation | Explanation |
|---|---|
| Local only | The tool and extension server run on one machine and are not hosted online. |
| Small RockYou sample | 12,669 passwords are used instead of the full ∼14 million. |
| Bloom false positives | About 0.1% of lookups may report a match that does not exist; real matches are never missed. |
| Limited extension testing | Tested on a few pages; not checked against all websites' security policies. |
| Exact matches only | Modified versions of leaked passwords are not detected, unlike MIGP. |
| Small, hand-selected test set | 25 passwords chosen by me, most of them breached by construction. The comparison shows that structure-only meters can miss leaked passwords, not how often they do. |
| Predictable patterns rated Strong | Name-or-place-and-year passwords score 3 or 4 in zxcvbn and are in no breach list, so the integrated tool rates them Strong. It inherits zxcvbn's blind spot for these. |
| Single machine and network | Speed figures were taken on one computer and one connection and show order of magnitude. |
| Targeted BreachCompilation example | The BreachCompilation-only result is a single case found by a targeted search, not a measured frequency. |
| Demo mode is not representative | Without the large datasets, the dictionary and Bloom filter checks use a small synthetic set. Only the full files reproduce the thesis results. |
| Usability untested | RQ3 depends on a user study that was not carried out. |

Table 5.4: Limitations and threats to validity.

# Chapter 6

# Conclusion

I set out to build an open-source tool that combines entropy analysis, pattern matching, dictionary checking and privacy-preserving breach detection. The five-check engine works with a verdict priority that reflects the finding that exposure matters more than structure. I scaled it to a billion-line breach collection by solving memory, speed and encoding problems, with measured evidence for each fix, and I delivered it through a web application and a browser extension that I tried on a real website. The code is public and runs without the large datasets, and it comes with automated tests. On a small test set, the integrated tool flagged every password known to be breached, while zxcvbn alone rated 22% of them as acceptable. This supports the claim that breach data adds information that structure-only scoring lacks. The same test showed that predictable passwords made of a name or place plus a year still pass as Strong. The contribution is the integration and its engineering, not a new technique. RQ1 is answered on a small set and needs a larger one before it can be generalised. The user study (RQ3) was not carried out, so the main research question is only partly answered.

## 6.1 Future Work

1. Host the tool online so that others can use it, and make the extension work on more websites.
2. Replace the RockYou sample with the full list.
3. Extend breach checking to modified versions of passwords, as MIGP does [7].
4. Grow the test set, ideally with a random sample of BreachCompilation, to estimate how often structure-only meters miss leaked passwords, and add a check for name-and-year patterns that zxcvbn rates highly.
5. Measure the delay of the HIBP network query under different network conditions.
6. Obtain ethics approval and run the usability study for RQ3.

# Appendix A

# Source Code Repository

The full source code of the application is available in a public GitHub repository:

`https://github.com/FaizanS27/pwbreach-tool`

The repository contains the source code of the checking engine, the Flask web application and the Chrome extension, the demonstration data files, the unit tests and the evaluation scripts. The README explains how to install, run and test the project. The repository also holds a copy of this thesis in PDF format and a screencast of the application working.

# Bibliography


[1] D. L. Wheeler, "zxcvbn: Low-budget password strength estimation," in *Proc. 25th USENIX Security Symp.*, Austin, TX, Aug. 2016, pp. 157–173.

[2] D. Malone and K. Maher, "Investigating the distribution of password choices," in *Proc. 21st Int. Conf. World Wide Web (WWW)*, Lyon, France, Apr. 2012, pp. 301–310.

[3] T. Hunt, "Understanding Have I Been Pwned's use of SHA-1 and k-anonymity," *Troy Hunt's Blog*, Jun. 30, 2022. [Online]. Available: `https://www.troyhunt.com/understanding-have-i-been-pwneds-use-of-sha-1-and-k-anonymity/`

[4] L. Sweeney, "k-anonymity: A model for protecting privacy," *Int. J. Uncertainty, Fuzziness Knowl.-Based Syst.*, vol. 10, no. 5, pp. 557–570, 2002.

[5] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970.

[6] D. Wang, X. Shan, Q. Dong, Y. Shen, and C. Jia, "No single silver bullet: Measuring the accuracy of password strength meters," in *Proc. 32nd USENIX Security Symp.*, Anaheim, CA, Aug. 2023, pp. 947–964.

[7] B. Pal *et al.*, "Might I get pwned: A second generation compromised credential checking service," in *Proc. 31st USENIX Security Symp.*, Boston, MA, Aug. 2022, pp. 1831–1848.

[8] J. Kim, M. Song, M. Seo, Y. Jin, and S. Shin, "PassREfinder: Credential stuffing risk prediction by representing password reuse between websites on a graph," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2024, pp. 1385–1404.

[9] J. Bonneau, "The science of guessing: Analyzing an anonymized corpus of 70 million passwords," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2012, pp. 538–552.

[10] M. Weir, S. Aggarwal, B. de Medeiros, and B. Glodek, "Password cracking using probabilistic context-free grammars," in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2009, pp. 391–405.

[11] W. Melicher *et al.*, "Fast, lean, and accurate: Modeling password guessability using neural networks," in *Proc. 25th USENIX Security Symp.*, Austin, TX, Aug. 2016, pp. 175–191.

[12] B. Ur *et al.*, "Design and evaluation of a data-driven password meter," in *Proc. ACM CHI*, Denver, CO, May 2017, pp. 3775–3786.

[13] K. Thomas *et al.*, "Protecting accounts from credential stuffing with password breach alerting," in *Proc. 28th USENIX Security Symp.*, Santa Clara, CA, Aug. 2019, pp. 1556–1571.

[14] L. Li, B. Pal, J. Ali, N. Sullivan, R. Chatterjee, and T. Ristenpart, "Protocols for checking compromised credentials," in *Proc. ACM CCS*, 2019, pp. 1387–1403.